Model Implementation

Group: MDS-IoT-Yu2 | Supervisor: Dr Yu Zhang

Sai Manoj Polina

Contents

1 Your Model **1**

2 Shared Settings — Fixed for All Models (Do Not Change) **2**

3 Data Setup **2**

4 Section 1 — Architecture **3**
4.1 1.1 Model Description ... 3
4.2 1.2 Layer Structure ... 3
4.3 1.3 Trainable Parameters ... 4
4.4 1.4 Architectural Choices and Justification ... 4

5 Section 2 — Training Setup **4**
5.1 2.1 Loss Function ... 4
5.2 2.2 Training Hyperparameters ... 5
5.3 2.3 Learning Rate Scheduling ... 5
5.4 2.4 Early Stopping ... 5
5.5 2.5 Training Difficulties ... 5

6 Section 3 — Results **6**
6.1 3.1 Quantitative Results ... 6
6.2 3.2 Predicted vs Ground Truth Trajectory ... 6
6.3 3.3 Loss Curve ... 6
6.4 3.4 MAE Breakdown by Scenario ... 7

7 Section 4 — Observations and Technical Reflection **7**
7.1 4.1 Where the Model Performs Well ... 7
7.2 4.2 Where the Model Struggles ... 7
7.3 4.3 Loss Curve Interpretation ... 8
7.4 4.4 Architecture vs Training — Root Cause of Limitations ... 8
7.5 4.5 One Change You Would Make ... 8

8 Submission Checklist **9**

---

1 Your Model

Model Name: *LSTM*

Your Name: *Sai Manoj Polina*

2 Shared Settings — Fixed for All Models (Do Not Change)

These settings are identical across all models to ensure fair comparison. Do not modify them.

Setting	Value
Input channels	Accelerometer x/y/z, Gyroscope x/y/z, Magnetometer x/y/z (9 total)
Window size	200 timesteps (1 second at 200Hz)
Sliding window stride	50 timesteps (75% overlap)
Sampling rate	200Hz
RONIN split	Train = train_dataset_1 + train_dataset_2, Val = seen_subjects_test_set, Test = unseen_subjects_test_set
Normalisation	Z-score per channel, fit on training set only
Output	sin(heading) and cos(heading) — 2 neurons
MAE calculation	min(|predicted - true|, 360 - |predicted - true|)
Platform	Jupyter Notebook

3 Data Setup

Follow these steps before writing any model code.

Step 1 — Download the RONIN dataset:

Download from: https://www.frdr-dfdr.ca/repo/dataset/816d1e8c-1fc3-47ff-b8ea-a36ff51d682a

You will get four folders:

- train_dataset_1
- train_dataset_2
- seen_subjects_test_set
- unseen_subjects_test_set

Step 2 — Clone the GitHub repository:

Clone from: https://github.com/AdhikariAnuj/Research-on-AI-Driven-IPS-Deep-Learning-for-Heading-Estimation-in-Pedestrian-Dead-Reckoning

Step 3 — Set up folder structure:

Place everything as follows:

```
project/
    preprocess.py
    your_model_notebook.ipynb
    train_dataset_1/
        a000_1/
            data.hdf5
            info.json
        ---
    train_dataset_2/
        ---
```

```
seen_subjects_test_set/
    ---
unseen_subjects_test_set/
    ---
```

Step 4 — Run the preprocessing script ONCE:

Open preprocess.py in Jupyter and run it. This will generate and save the following files to disk:

- X_train.npy, y_train.npy
- X_val.npy, y_val.npy
- X_test.npy, y_test.npy
- mean.npy, std.npy

This may take several minutes due to the size of the training data. You only need to run this once — after that the .npy files are saved permanently.

Step 5 — Load data in your model notebook:

At the top of every model notebook paste this loading snippet:

```
import numpy as np

X_train = np.load('X_train.npy')
y_train = np.load('y_train.npy')
X_val = np.load('X_val.npy')
y_val = np.load('y_val.npy')
X_test = np.load('X_test.npy')
y_test = np.load('y_test.npy')
```

Do not modify the preprocessing script. If you encounter issues contact Anuj.

---

4 Section 1 — Architecture

4.1 1.1 Model Description

This model uses a two-layer LSTM network to learn temporal patterns from sequential inertial sensor data (accelerometer, gyroscope, and magnetometer). The input consists of 200-timestep windows with 9 channels, and the model predicts the heading direction by outputting sine and cosine values. The LSTM captures time dependencies in motion data to estimate orientation accurately over time.

4.2 1.2 Layer Structure

List your full layer structure in order, be specific and exact. Use the relevant format below for your model type and delete the others.

For LSTM:

Layer	Details
Input	(200 x 9)
LSTM Layer 1	Hidden size: 128
Dropout	Rate: 0.3
LSTM Layer 2	Hidden size:128
Fully Connected	128 -> 2 output neurons

4.3 1.3 Trainable Parameters

Total trainable parameters: *203522*

4.4 1.4 Architectural Choices and Justification

*The architectural design of the LSTM model was guided by the need to effectively capture temporal dependencies in sequential inertial sensor data for heading estimation. A **two-layer LSTM** was chosen instead of a single layer because deeper recurrent networks are better at learning hierarchical temporal representations, where lower layers capture short-term motion dynamics and higher layers capture longer-term dependencies. This is consistent with findings from Graves 2013 Speech Recognition RNN paper, which showed that stacked recurrent layers improve sequence modelling performance.*

A hidden size of 128 was selected as a balance between model capacity and computational efficiency. Larger hidden sizes can capture more complex patterns but increase the risk of overfitting and training instability, especially with noisy IMU data. Prior work in inertial navigation and sequence modelling commonly uses hidden sizes in the range of 64–256, making 128 a reasonable mid-point that provides sufficient representational power without excessive computational cost.

A dropout rate of 0.3 was introduced between LSTM layers to reduce overfitting, as sequential sensor data often contains high temporal correlation. This choice is supported by Srivastava et al. 2014 Dropout paper, which demonstrates that dropout improves generalisation in deep neural networks.

The model uses a fully connected output layer mapping to two neurons (sin and cos of heading) instead of directly predicting angles. This design avoids discontinuities at the 0°/360° boundary and enables stable learning of circular quantities, a common approach in orientation estimation tasks.

Unlike transformer-based models, no attention heads are used because LSTM networks inherently model sequential dependencies through recurrent connections, making them more suitable for relatively shorter sequences (200 timesteps) where long-range attention is less critical.

Overall, the architecture was chosen to balance temporal modelling capability, robustness to noise, and computational efficiency, making it appropriate for real-world inertial heading estimation.

---

5 Section 2 — Training Setup

5.1 2.1 Loss Function

*The loss function used is **Mean Squared Error (MSE)** applied to the predicted and true sine and cosine values of the heading angle. Instead of directly predicting angles, the model predicts sin(θ) and cos(θ), which avoids discontinuity issues at the 0°/360° boundary.*
Using MSE on sine and cosine ensures smooth optimisation in a continuous space, allowing the model to learn circular relationships effectively

5.2 2.2 Training Hyperparameters

Hyperparameter	Value
Optimizer.	Adam
Learning rate	0.001
Batch size	64
Number of epochs	30

5.3 2.3 Learning Rate Scheduling

A learning rate scheduler (ReduceLROnPlateau) was used to adjust the learning rate based on validation loss. When the validation loss stopped improving for 3 consecutive epochs, the learning rate was reduced by a factor of 0.5. This helps the model converge more effectively and prevents oscillations during training.

5.4 2.4 Early Stopping

Yes, early stopping was used with a patience value of 5 epochs. The validation loss was monitored, and training stopped at epoch 14 when the validation loss did not improve for 5 consecutive epochs. This helps prevent overfitting and reduces unnecessary training time.

5.5 2.5 Training Difficulties

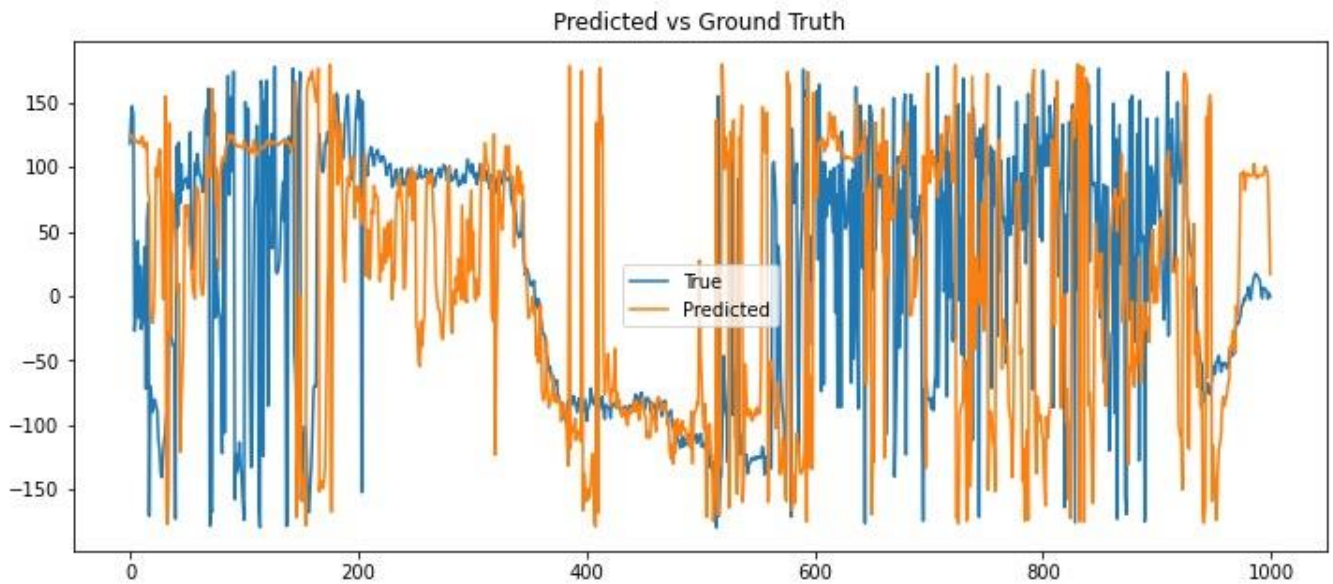
During training, minor fluctuations in validation loss were observed due to noise in the sensor data. This was addressed using dropout and learning rate scheduling to stabilise training. No major convergence issues were encountered.

6 Section 3 — Results

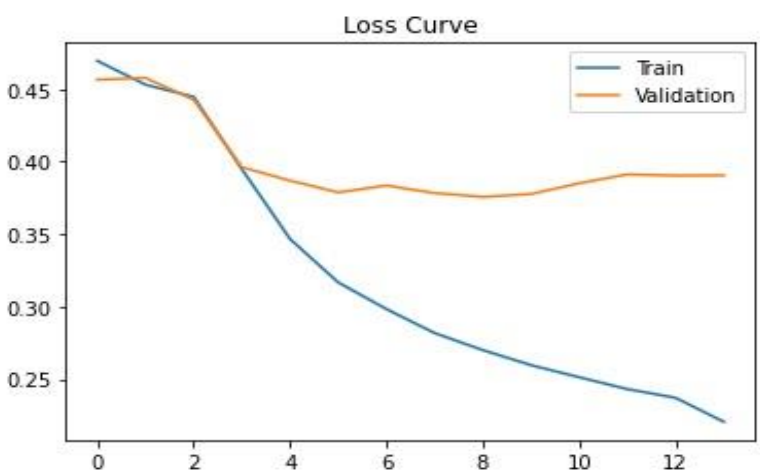
6.1 3.1 Quantitative Results

Metric	Value
MAE on test set (degrees)	67.05 degrees
RMSE on test set (degrees	83.61 degrees
Approximate training time	345.76 minutes (5.8 hours)

6.2 3.2 Predicted vs Ground Truth Trajectory)



6.3 3.3 Loss Curve



6.4 3.4 MAE Breakdown by Scenario

```
MAE Breakdown (degrees):
Straight walking: 59.96°
Sharp turns: 86.07°
Short trajectories: 69.03°
Long trajectories: 65.08°
```

7 Section 4 — Observations and Technical Reflection

7.1 4.1 Where the Model Performs Well

The model performs relatively well in scenarios involving smooth and consistent motion, particularly during straight walking. This is supported by the scenario-based results, where the Mean Absolute Error (MAE) for straight walking is 59.96°, which is lower than the error observed during sharp turns (86.07°). In the predicted versus ground truth plot, the model is able to follow the general trend of the true trajectory during smoother segments, indicating that it captures underlying motion patterns effectively.

This performance can be attributed to the LSTM architecture. The LSTM layers are designed to capture temporal dependencies in sequential data, allowing the model to learn patterns over time. The use of a window size of 200 timesteps provides sufficient temporal context for modelling consistent motion. Additionally, representing the output as sin(θ) and cos(θ) helps avoid discontinuity issues in angular prediction, enabling more stable learning. Overall, the model performs well in predictable, low-noise conditions where motion changes gradually.

7.2 4.2 Where the Model Struggles

The model struggles significantly in scenarios involving sharp turns and highly dynamic motion. This is evident from the higher MAE of **86.07°** in sharp turning scenarios. In the predicted versus ground truth plot, the model shows large fluctuations and instability, with frequent deviations from the true trajectory. The predictions appear noisy and inconsistent, especially during rapid changes in direction.

Furthermore, the errors for short trajectories **(69.03°)** and long trajectories **(65.08°)** remain relatively high, indicating that the model has difficulty maintaining accuracy across both short-term and long-term sequences. These limitations arise because LSTM networks, while effective for sequential data, have limited capability in capturing sudden transitions. Additionally, the inertial sensor data is inherently noisy, which

further affects prediction accuracy. As a result, the model struggles in complex motion scenarios where rapid and unpredictable changes occur.

7.3 4.3 Loss Curve Interpretation

The loss curve shows that the training loss decreases steadily from **0.4695** to **0.2206**, indicating that the model is successfully learning from the training data. The validation loss initially decreases and reaches its lowest value around epoch 9 (approximately 0.3757), after which it begins to stabilise and slightly increase.

This pattern suggests that the model is learning meaningful features during the early stages of training. However, after a certain point, the model begins to overfit the training data, as the training loss continues to decrease while the validation loss stops improving. Early stopping was triggered at epoch 14, which helped prevent further overfitting and unnecessary computation. Overall, the training process is stable, but the model's ability to generalise to unseen data is limited.

7.4 4.4 Architecture vs Training — Root Cause of Limitations

The limitations observed in the model performance are primarily due to both architectural and data-related factors, with the architecture playing a more significant role. The relatively high MAE **(67.05°)** and RMSE **(83.61°)** indicate that the model struggles to accurately estimate heading, particularly in complex motion scenarios.

From an architectural perspective, LSTM processes data sequentially but lacks mechanisms to focus on the most important parts of the input sequence. This makes it less effective in handling long-range dependencies and sudden directional changes. Additionally, the model capacity, defined by its hidden size and number of layers, may not be sufficient to capture the full complexity of the motion patterns present in the dataset.

From a training and data perspective, inertial measurement unit (IMU) data is inherently noisy, and variations between different subjects and environments can negatively impact generalisation. Furthermore, the model relies only on raw sensor inputs, which may limit its ability to extract higher-level features. Overall, while the training process is stable, the architecture is the main bottleneck affecting performance.

7.5 4.5 One Change You Would Make

The limitations observed in the model performance are primarily due to both architectural and data-related factors, with the architecture playing a more significant role. The relatively high MAE (67.05°) and RMSE (83.61°) indicate that the model struggles to accurately estimate heading, particularly in complex motion scenarios.

From an architectural perspective, LSTM processes data sequentially but lacks mechanisms to focus on the most important parts of the input sequence. This makes it less effective in handling long-range dependencies and sudden directional changes.

Additionally, the model capacity, defined by its hidden size and number of layers, may not be sufficient to capture the full complexity of the motion patterns present in the dataset.

From a training and data perspective, inertial measurement unit (IMU) data is inherently noisy, and variations between different subjects and environments can negatively impact generalisation. Furthermore, the model relies only on raw sensor inputs, which may limit its ability to extract higher-level features. Overall, while the training process is stable, the architecture is the main bottleneck affecting performance.

---

8 Submission Checklist

Before submitting make sure you have completed the following:
□ Model name and your name filled in at the top
□ All sections completed — no empty fields
□ MAE and RMSE reported in degrees using the correct circular formula
□ Predicted vs ground truth trajectory plot included
□ Loss curve plot included (deep learning models only)
□ Code submitted as a separate .ipynb file
□ Preprocessing script was not modified

Submit your completed report and code to: **:** https://github.com/AdhikariAnuj/Research-on-AI-Driven-IPS-Deep-Learning-for-Heading-Estimation-in-Pedestrian-Dead-Reckoning

Internal group deadline: April 23, 2026 — Hard deadline, no exceptions. Final submission to
Dr Yu Zhang is May 1, 2026.